ІНЖЕНЕРНА КНИГА

Зовнішні результати

Код. Дослідження. Зовнішній наслідок.

DiadkoShmek

4 власні PR merged; 2 внески у merged fixes мейнтейнерів; 1 заміна ще OPEN; 1 аудит. 3 часові записи про 2 арени/події, без подвійного підрахунку Zoria.

Датований запис із публічними першоджерелами.

Зміст

Роль, метод і одиниця доказу 3

01 FilesDrop 4

02 Prime Intellect 6

03 M7 MCP 8

04 Braintrust SDK JavaScript 10

05 Kraft 12

06 PyTorch TensorDict 14

07 BlackJAX 16

08 Apache TVM-FFI 18

09 CodinGame - Mad Pod Racing 20

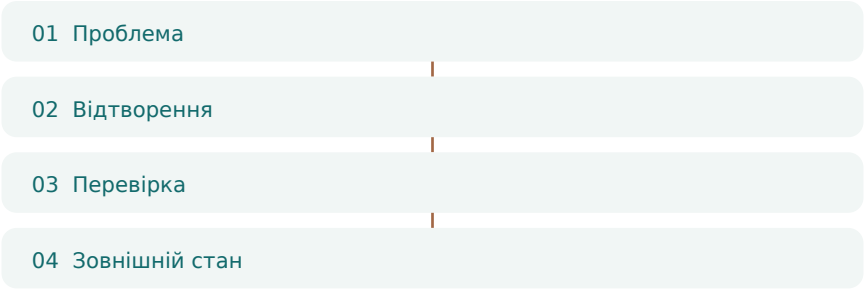
10 Zoria 2026 / UA SKILLS 22

Як зростає журнал 24

Роль, метод і одиниця доказу

Я досліджую дефект, визначаю контракт і перевіряю наслідок. AI-інструменти допомагають із пошуком, реалізацією та перевіркою; я задаю напрям, відповідаю за інженерне рішення й погоджую зовнішні дії. Ця книга не вимірює відсоток автономності чи перевагу однієї моделі.

4 власні PR merged; 2 внески у merged fixes мейнтейнерів; 1 заміна ще OPEN; 1 аудит. 3 часові записи про 2 арени/події, без подвійного підрахунку Zoria.



Локальний тест перевіряє механізм. Upstream merge або credit підтверджує зовнішній наслідок. Жоден із них сам собою не доводить повний CI, adoption чи оплату.

Дати мають UTC-прив’язку. Для історичних зрізів збережені початкові часові поля; дата перевірки оголошення може відрізнятися від дати публікації.

01 FilesDrop

2026-10-21 · Власний PR merged

Проблема

Я почав із двох різних порушень життєвого циклу: невдале видалення зникало з наступної партії cleanup, а deadline не припиняв очікування DNS. У першому випадку важлива не сама помилка диска, а втрата можливості повторити роботу. У другому - межа між очікуванням запиту та системним resolver.

Рішення та роль

Я зберіг невдалі елементи для наступного проходу й відділив успішні видалення. Контрприклад на 501 файл показав, що одна постійна відмова може блокувати наступну партію. Для URL-upload сигнал скасування обмежує очікування, зберігаючи перевірки адрес та redirect-поведінки.

Перевірка

Регресії спершу падали на базі. У збереженому локальному прогоні пройшли 236 тестів у 12 API suites. Тимчасові файли та DNS/HTTP doubles перевіряли потрібні переходи стану, не поведінку production-сервісу.

Що переношу далі

Cleanup має зберігати можливість повторення. Скасування очікування не означає зупинку системної операції.

Межа зовнішнього результату

Злиті API-виправлення з локальними регресійними перевірками. Розгортання у робочому середовищі, перевірка живого сервісу чи оплата цим записом не підтверджені.

Джерела

1. Першоджерело

2. Докладний кейс

02 Prime Intellect

2026-10-24 · Внесок у merged fix мейнтейнера

Проблема

Я досліджував, як конфігурація моделі й клієнта успадковує загальні sampling-параметри. Інший учасник уже подав початковий issue. Моя задача полягала в тому, щоб перетворити опис на перевірюваний контракт і зрозумілий відтворювач.

Рішення та роль

Я запропонував межу на рівні явного model/client override. Мейнтейнер обрав точнішу межу клієнта: інша модель на тому самому сервері зберігає спільні параметри, а окремий клієнт отримує власні. Це інша реалізація й дещо інший контракт, а не merge мого патчу.

Перевірка

Матриця моєї пропозиції перейшла від 11 failing і 7 passing сценаріїв до 18 passing. Це число сценаріїв, не кількість дефектів. Публічна відповідь мейнтейнера окремо визнає роль відтворення й дослідження. Живе inference або навчання я цими тестами не підтверджував.

Що переношу далі

Корисний внесок може змінити рішення мейнтейнера, навіть коли власний patch не стає прийнятим кодом.

Межа зовнішнього результату

Мейнтейнер відзначив наше відтворення й дослідження. Наш патч замінено, а не злито; початкову проблему зареєстрував інший учасник. Працевлаштування або оплата не заявляються.

Джерела

1. Першоджерело

2. Наша пропозиція

3. Реалізація мейнтейнера

4. Реалізація мейнтейнера

5. Докладний кейс

03 M7 MCP

2026-10-29 · Аудит відзначено в релізі

Проблема

Я перевіряв конкретний snapshot MCP-сервера: переходи фаз, повторні виклики, збереження стану та передачу роботи. Аудит вимагав відділити дефект контракту від помилки мого спостереження та від обмеження середовища.

Рішення та роль

Я сформував відтворювані спостереження з причиною, входом і межею висновку. Мейнтейнери створили власний реєстр проблем і реалізували перший maintenance release. Їхні записи не є взаємно-однозначною копією груп мого аудиту.

Перевірка

Оригінальний публічний PDF збережено без переписування під пізніший реліз. Focused follow-up на зафіксованому commit пройшов 168 тестів і SDK stdio-перевірку, але зберіг три незакриті error-path сценарії. Це не повний повторний аудит.

Що переношу далі

Аудит має залишати перевірюваний маршрут до помилки та не приховувати те, чого наступний реліз ще не довів.

Межа зовнішнього результату

Публічне відзначення перевірки й класифікації проблем та випуск мейнтейнерами релізу з виправленнями. Групи аудиту та їхні записи реєстру не взаємозамінні. Прицільна повторна перевірка зберегла незакриті сценарії помилок; живі моделі й IDE не перевірялися.

Джерела

1. Першоджерело

2. Докладний кейс

04 Braintrust SDK JavaScript

2026-11-02 · Власний PR merged · 2026-11-02T11:37:30Z

Проблема

Проблема виникала на межі чанків логування: багатобайтовий UTF-8 символ може бути розділений між двома порціями байтів. Коректний текст не повинен залежати від того, як транспорт розбив потік.

Рішення та роль

Я зберіг стан decoder між читаннями й відділив завершення потоку від проміжного чанка. Перевірка кожної byte boundary перетворила загальний опис пошкоджених символів на точний інваріант: різні розбиття того самого потоку дають той самий текст.

Перевірка

Початковий focused прогін мав шість відмов і 13 passing controls. Пов’язані suites пройшли 178 тестів на обох перевірених версіях Node. Це локальні перевірки визначених шляхів, не повний monorepo, browser чи реальний provider end-to-end.

Що переношу далі

Межа чанка належить транспорту. Стан текстового decoder має належати всьому логічному потоку.

Межа зовнішнього результату

Злите виправлення потокового логування UTF-8 з локальними регресіями. Повний монорепозиторій, браузер і наскрізний сценарій із реальним провайдером не перевірялися.

Джерела

1. Першоджерело

2. Докладний кейс

05 Kraft

2026-11-02 · Власний PR merged · 2026-11-02T12:08:30Z

Проблема

Повторний user-scope init бачив уже наявну MCP-реєстрацію й завершувався раніше, ніж оновлював skills. Команда виглядала повторюваною, але її друга частина фактично не виконувалася.

Рішення та роль

Я розділив успішний стан уже наявної реєстрації та справжню помилку subprocess. Перший дозволяє перейти до оновлення skills; друга має зупинити процес до записів. Це виправлення порядку дій, без виклику живого агента.

Перевірка

Focused suite пройшов 16 тестів. Репозиторні ci-checks та mutation checks перевіряли розрізнення гілок. Claude subprocess був test double; користувацькі конфігурації й зовнішній агент у цих перевірках не змінювалися.

Що переношу далі

Повторний init має повторювати корисну роботу, а не плутати idempotent стан із відмовою.

Межа зовнішнього результату

Злите виправлення повторної користувацької ініціалізації. У тестах підпроцес Claude замінено імітацією: без живих викликів агента чи змін користувацьких конфігурацій.

Джерела

1. Першоджерело

2. Докладний кейс

06 PyTorch TensorDict

2026-11-01 · Внесок у merged fix мейнтейнера

Проблема

Частковий checkpoint у вкладеному TensorDictParams міг відмовити навіть із strict=False. Я перевіряв не лише пропущений ключ, а й вкладені пропуски, assignment і pre-hooks - поведінку, на яку вже покладаються користувачі PyTorch.

Рішення та роль

Я запропонував loader fix і differential controls проти native nn.Module. Мейнтейнер обрав менший owner-level крок: делегувати власному loader PyTorch замість підтримки копії. Публічний credit стосується попереднього fix, report і thorough tests.

Перевірка

Наш кандидат мав 44 failing сценарії та чотири passing controls на базі; focused suites після нього дали 251 pass і два CUDA skips. Ці числа не описують фінальну реалізацію мейнтейнера. Її прийняття фіксує окремий upstream PR.

Що переношу далі

Коли контракт уже належить нижчому шару, делегування може бути сильнішим за точнішу копію.

Межа зовнішнього результату

Мейнтейнер відзначив попереднє виправлення, звіт і ретельні тести. Наш PR #3426 замінено, не злито; менша прийнята реалізація належить мейнтейнеру.

Джерела

1. Першоджерело

2. Наша пропозиція

3. Відзначення внеску

4. Докладний кейс

07 BlackJAX

2026-11-02 · Власний PR merged · 2026-11-02T14:43:52Z

Проблема

Barker MCMC міг відхилити крок до області вищої ймовірності через переповнення проміжного exp. Кінцевий log-expression лишався математично скінченним, але прямий порядок обчислення втрачав його.

Рішення та роль

Я замінив нестабільний log(1+exp(x)) на стандартний softplus. Важливий доказ - не просто відсутність inf, а правильна proposal probability відносно незалежного обчислення густини. Звичайні значення залишилися контрольними випадками.

Перевірка

Незалежне обчислення NumPy на 768 CPU-кроках знаходило 142 розбіжності на базовій версії й нуль після зміни; всі 384 звичайні контрольні випадки проходили й раніше. Повний набір тестів не запускався; незмінений збій CI GIST відтворено на базовій та зміненій версіях. Роботу на GPU/TPU та приріст швидкодії не заявлено.

Що переношу далі

Стабілізувати треба проміжний розрахунок, зберігши математичну величину та контрольні режими.

Межа зовнішнього результату

Злите виправлення чисельної стабільності Barker MCMC з порівняльними перевірками на CPU. Повний набір тестів не запускався; незмінений збій CI GIST відтворено на точних базовій і зміненій версіях. Роботу на GPU/TPU та приріст швидкодії не заявлено.

Джерела

1. Першоджерело

2. Докладний кейс

08 Apache TVM-FFI

2026-11-02 · Superseded; заміна OPEN

Проблема

Рівні короткі byte-значення з inline та heap storage отримували різні structural hashes. Це порушувало очікування StructuralKey cache lookup і native map deduplication. Мій patch виправляв обидва шляхи структурного hashing.

Рішення та роль

Мейнтейнер публічно пов’язав нашу пропозицію з наступним PR та обрав інший шар: короткий moved std::string одразу створює inline String/Bytes. Набори змінених файлів різні; прямого перенесення мого патчу у порівняних diff не виявлено.

Перевірка

Зіставлено нативні стани PR, chronology, коміти та diff. Constructor-side fix прибирає producer path, який використовував початковий reproducer. Наступну реалізацію та всі explicit external-heap випадки для цього запису повторно не виконували.

Що переношу далі

Відтворення може привести до іншого виправлення. Superseded та merged - різні зовнішні наслідки.

Межа зовнішнього результату

Мейнтейнер змінив конструктор короткого std::string rvalue, а не переніс наш патч хешування. Наш PR не злито; заміна лишалася OPEN. Її повну поведінку й усі сценарії з явним зберіганням у купі для цього запису повторно не виконували.

Джерела

1. Першоджерело

2. Наша пропозиція

3. Реалізація мейнтейнера

4. Докладний кейс

09 CodinGame - Mad Pod Racing

2026-10-21 (Europe/Helsinki) · Датований зріз арени · UTC 2026-10-20T22:06:15.975Z · 86/1657 (Legend League)

Проблема

Я будував рішення для безперервної арени, де локальна перемога не гарантує зовнішнього місця. Для розбору збережено дві історії: швидке відхилення гіпотези й умовний локальний кандидат після зміни критеріїв.

Рішення та роль

V79 зупинено після 2 перемог у восьми завершених іграх: потрібна межа вже стала недосяжною. V89b дав 221/400, але критерії змінювалися після попередніх відмов, тому я не прирівнюю це до чистої незалежної перемоги чи поданої версії.

Перевірка

Зовнішній league snapshot має окреме публічне джерело та датований screenshot. Локальна витяжка не містить candidate code, referee і повних traces; самі hashes не дозволяють повторити матчі чи незалежно засвідчити їхній вміст.

Що переношу далі

Стоп-критерій захищає від доведення бажаного результату. Локальний експеримент і зовнішня арена мають різні одиниці доказу.

Межа зовнішнього результату

Датований зріз місця в лізі, не фінальний результат турніру й не найкраще місце за весь час.

Джерела

1. Першоджерело

2. Профіль арени

10 Zoria 2026 / UA SKILLS

2026-10-19 (Europe/Helsinki) · Історичний попередній результат · UTC 2026-10-20T22:06:05.463Z · 5/9 (загальний попередній підсумок учасників другого етапу)

2026-10-27 (Europe/Helsinki) · Призер сезону; точне місце не встановлено

Проблема

Цей кейс зберігає дві часові точки одного турніру. Попередня таблиця другого етапу та пізніше оголошення організатора описують різні рівні завершеності. Я не додаю їх як два незалежні турнірні результати.

Рішення та роль

Моя інженерна роль полягала в розробці й перевірці власного ігрового рішення та збереженні меж зовнішнього доказу. Інженерний розбір коду ще не опубліковано, тому книга не підміняє його припущеннями про алгоритм чи автономність.

Перевірка

Попередня таблиця не включала R3. Пізніше організатор назвав обліковий запис серед призерів. Дата перевірки оголошення не є датою його публікації; точне фінальне місце й виплата цим записом не встановлені.

Що переношу далі

Історія повинна зберігати попередній стан, не перетворюючи його на ще одну перемогу.

Межа зовнішнього результату

Публічна таблиця не враховує епізод R3; цей запис не встановлює фінального місця, нагороди або виплати.

Організатор називає цей обліковий запис серед п’яти призерів. Запис не встановлює точного фінального місця, кількості ранжованих учасників або виплати.

Джерела

1. Першоджерело

2. Першоджерело

Як зростає журнал

Для нового результату я зберігаю дату спостереження UTC, свою роль, первинне джерело й точний зовнішній статус. Старе відтворення лишається датованим; повторну перевірку не видаю за повний аудит. Додаю запис до results.json, розбір до book/narrative.json і запускаю збірку та офлайн-перевірку.

Джерела збірки

results.json:
e40835983abddcb4f400bee44e4cc486d16165b97cbd06d7a4c54051cc01cd07

book/narrative.json:
ca3bca8162d1d591bcc6ec4b662c919ea5c69935a179ff4abb19424c8f67b5f3

Детальні двомовні звіти та незмінений історичний аудит M7 доступні в репозиторії.

github.com/DiadkoShmek/engineering-results